

# US Patent & Trademark Office

## Patent Public Search | Text View

United States Patent  
Kind Code  
Date of Patent  
Inventor(s)

12412129  
B2  
September 09, 2025  
Ma; Lichuan et al.

### Distributed support vector machine privacy-preserving method, system, storage medium and application

#### Abstract

A distributed support vector machine privacy-preserving method includes: dividing a secret through secret sharing among all participating entities, iteratively exchanging a part of the information divided by the participating entities, and solving sub-problems locally; performing an iteration until a convergence is reached to find a global optimal solution; and in consideration of the generality of the privacy-preserving method, adopting a privacy-preserving method based on a vertical data distribution and a privacy-preserving method based on a horizontal data distribution, respectively; wherein the participating entities do not trust each other, and interact through a multi-party computation for local training. The method is applied to an honest-but-curious scenario, and uses the idea of data division to perform local computation through the interaction of part of the data among users to finally reconstruct the secret to preserve data privacy.

**Inventors:** Ma; Lichuan (Xi'an, CN), Pei; Qingqi (Xi'an, CN), Huang; Zijun (Xi'an, CN)  
**Applicant:** Xidian University (Xi'an, CN); Xi'an Xidian Blockchain Technology CO., Ltd. (Xi'an, CN)  
**Family ID:** 75701736  
**Assignee:** XIDIAN UNIVERSITY (Xi'an, CN); XI'AN XIDIAN BLOCKCHAIN TECHNOLOGY CO., LTD. (Xi'an, CN)  
**Appl. No.:** 17/317908  
**Filed:** May 12, 2021

#### Prior Publication Data

| Document Identifier | Publication Date |
|---------------------|------------------|
| US 20220237519 A1   | Jul. 28, 2022    |

#### Foreign Application Priority Data

|    |                |               |
|----|----------------|---------------|
| CN | 202110054339.X | Jan. 15, 2021 |
|----|----------------|---------------|

#### Publication Classification

**Int. Cl.:** G06N20/10 (20190101); G06N7/00 (20230101)

**U.S. Cl.:**

**CPC** G06N20/10 (20190101); G06N7/00 (20130101);

#### Field of Classification Search

**CPC:** G06F (30/27); G06N (20/10); G06N (7/00)

#### References Cited

##### U.S. PATENT DOCUMENTS

| Patent No.   | Issued Date | Patentee Name | U.S. Cl. | CPC          |
|--------------|-------------|---------------|----------|--------------|
| 2018/0218171 | 12/2017     | Bellala       | N/A      | G06F 21/6254 |
| 2021/0209247 | 12/2020     | Mohassel      | N/A      | A63B 21/023  |

##### OTHER PUBLICATIONS

Forero et.al, Consensus-based distributed linear support vector machines, Apr. 12, 2010, in Proceedings of the 9th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN '10) (Year: 2010). cited by examiner  
Jia et.al, Jia Q, Guo L, Fang Y, Wang G. Efficient Privacy-preserving Machine Learning in Hierarchical Distributed System, Jul. 24, 2018, IEEE Trans Netw Sci Eng (Year: 2018). cited by examiner  
Snyder et.al, Yao's Garbled Circuits: Recent Directions and Implementations, Apr. 14, 2014, University of Illinois at Chicago (Year: 2014). cited by examiner  
Demmler et.al, ABY—A Framework for Efficient Mixed-Protocol Secure Two-Party Computation, Feb. 7, 2015 (Year: 2015). cited by examiner  
Xu et.al, Privacy-Preserving Machine Learning Algorithms for Big Data Systems, Jul. 23, 2015, 2015 IEEE 35th International Conference on

Primary Examiner: Shmatov; Alexey

Assistant Examiner: Diep; Duy T

Attorney, Agent or Firm: Bayramoglu Law Offices LLC

---

## Background/Summary

### CROSS REFERENCE TO THE RELATED APPLICATIONS

(1) This application is based upon and claims priority to Chinese Patent Application No. 202110054339.X, filed on Jan. 15, 2021, the entire contents of which are incorporated herein by reference.

### TECHNICAL FIELD

(2) The present invention pertains to the technical field of data privacy-preserving, and more particularly, to a distributed support vector machine privacy-preserving method, system, storage medium and application.

### BACKGROUND

(3) Today's information age is now witnessing the explosive growth of data. As the scale of computer systems becomes larger and larger, distributed processing methods are increasingly accepted by the related industry. Moreover, machine learning algorithms have been applied to various fields. In this case, since the distributed processing method can handle a larger amount of samples, it can better exploit the advantages of machine learning algorithms so that the algorithms can be applied on a large scale. The support vector machine is one of the most widely used machine learning algorithms. Prior studies generally use alternating direction method of multipliers (ADMM) algorithms to solve machine learning optimization problems such as optimization problems of support vector machines. Meanwhile, data used for training are owned by multiple entities, but the sharing and training of the data are hindered due to the sensitivity of the data. Most distributed algorithms require each node to explicitly exchange and disclose state information to its neighboring node in each iteration. This means that many practical distributed applications face serious privacy issues. In this regard, it is unacceptable to merely save the original data locally for privacy-preserving, and it is also necessary to preserve the privacy of the interactive parameters in the process of implementing the distributed ADMM algorithm. Herein, an ADMM algorithm-based privacy-preserving technique will be presented based on a support vector machine scenario.

(4) However, existing research on privacy-preserving of support vector machine scenarios still faces challenges to be urgently solved in terms of privacy and accuracy. Two methods are commonly used to realize privacy-preserving in distributed optimization algorithms. The first method is a perturbation method, which mainly uses the technique of differential privacy. This method is highly efficient, but introduces noise and thus will cause the loss of data availability and impair the accuracy of the optimization results. Although the related studies have made a balance between privacy and accuracy, the speed of convergence to the optimal classifier will always slow down. The second method is a cryptographic method, including secure multi-party technology and homomorphic encryption. The homomorphic encryption method has excessively high computational overhead and is thus difficult to apply to practical applications. Additionally, in the current research, most of the support vector machine privacy-preserving scenarios involve only distributed deployment of data and single-machine processing without considering the privacy leakage problem of information interaction during the collaborative training of the fully distributed support vector machine algorithm with multiple machines and multiple data sources. Some research work has focused on the privacy leakage problem, but has not fully resolved the horizontal and vertical distribution of data.

(5) As analyzed above, the prior art has the following problems and shortcomings. The existing distributed support vector machines have mutually reciprocal shortcomings between computational overhead and security. That is, a high-security method has the problem of high computational overhead, whereas a high-efficiency method has security issues. In addition, such methods must give consideration to both the machine learning scenario and the accuracy of the training results.

(6) The difficulty of solving the above-mentioned problems and shortcomings is: to solve the privacy problem of the interactive computation of the intermediate states in the machine learning training process. Although homomorphic encryption can perform multi-party secure computation, it incurs high computational complexity.

(7) To solve the above-mentioned problems and shortcomings, it is highly desirable to provide a high-efficiency method capable of simultaneously ensuring the security of multi-party computation when processing data for machine learning training to achieve the same effectiveness as homomorphic encryption without substantial overhead, thereby preserving data privacy based on the premise of an ensured accuracy of the training results.

### SUMMARY

(8) In view of the problems in the prior art, the present invention provides a distributed support vector machine privacy-preserving method, system, storage medium and application.

(9) The present invention is achieved by adopting the following technical solutions. A distributed support vector machine privacy-preserving method includes: dividing a secret through a secret sharing among all participating entities, iteratively exchanging a part of information divided by the participating entities, and solving sub-problems locally; performing an iteration until a convergence is reached to find a global optimal solution; and in consideration of the generality of the privacy-preserving method, adopting a privacy-preserving method based on a vertical data distribution and a privacy-preserving method based on a horizontal data distribution, respectively; wherein the participating entities do not trust each other, and interact through a multi-party computation for local training.

(10) Further, the distributed support vector machine privacy-preserving method specifically includes:

(11) step 1: establishing a network communication environment with a plurality of data sources;

(12) step 2: choosing a support vector machine scenario with a vertical distribution or a horizontal distribution according to a data distribution of the data sources;

(13) step 3: allowing all participating entities to solve the sub-problems locally;

(14) step 4: allowing all participating entities to use a Boolean sharing to split a penalty parameter and exchange a part of the penalty parameter with a neighboring node to update the parameter;

(15) step 5: allowing all participating entities to use an arithmetic sharing to split an updated iterative variable and exchange a part of the updated iterative variable with the neighboring node to compute a Lagrange parameter in a shared form;

(16) step 6, allowing all participating entities to reconstruct the secret;

(17) step 7, returning to step 3 if the iteration does not reach the convergence; and

(18) step 8, outputting a training result.

(19) Further, an objective function of the horizontal distribution and an objective function of the vertical distribution in step 2 are respectively:

$$(20) \min_{v_i} \frac{1}{2} \sum_{i=1}^N \text{Math. } v_i^T (I_{D+1} - I_{D+1}) v_i + NC \sum_{i=1}^N \text{Math. } 1_i^T s.t. \{ Y_i B_i V_i \geq 1_i - i, i \geq 0 (i=1, \text{Math.}, N) \};$$

$$AV = 0$$

and

(21) wherein  $v_{\text{sub}.i} = [\omega_{\text{sub}.i.\text{sup}.T, b_{\text{sub}.i}.\text{sup}.T}]^T$ ,  $V = [v_{\text{sub}.1.\text{sup}.T}, \dots, v_{\text{sub}.N.\text{sup}.T}]^T$ ,  $B_{\text{sub}.i} = [X_{\text{sub}.i,1.\text{sub}.i.\text{sup}.T}, 1_{\text{sub}.i} \in \mathbb{R}^{\text{sup}.1 \times M}]$ ,  $X_{\text{sub}.i}$  is an  $i_{\text{sup}.th}$  participant and a  $j_{\text{sup}.th}$  participant,  $X_{\text{sub}.i} = [x_{\text{sub}.i,1.\text{sup}.T}, \dots, x_{\text{sub}.i,M.\text{sup}.T}]^T$ ,  $Y_{\text{sub}.i} = \text{diag}(y_{\text{sub}.i,1}, \dots, y_{\text{sub}.i,M})$ ,  $y_{\text{sub}.i,j}$  is a  $j_{\text{sup}.th}$  data entry and a corresponding label,  $\xi_{\text{sub}.i} = [\xi_{\text{sub}.i,1}, \dots, \xi_{\text{sub}.i,M}]$ ,  $N$  is a number of participants, and  $M$  is a number of training set samples for each participant of the participants; and

$$(22) \min_{v_i} \text{Math. } \frac{1}{2} v_i^T (I_{D+1} - I_{D+1}) v_i + 1_M^T (1_M - Y \sum_{i=1}^N \text{Math. } B_i v_i) s.t. z_i = B_i v_i, i = 1, \text{Math.};$$

(23) wherein  $v_{\text{sub}.i} = [\omega_{\text{sub}.i.\text{sup}.T, b_{\text{sub}.i}.\text{sup}.T}]^T$ ,  $B_{\text{sub}.i} = [X_{\text{sub}.i,1.\text{sub}.M}]$ ,  $1_{\text{sub}.M} \in \mathbb{R}^{\text{sup}.M \times 1}$ ,  $Y_{\text{sub}.i} = \text{diag}(y_{\text{sub}.i,1}, \dots, y_{\text{sub}.i,M})$ ,  $y_{\text{sub}.j}$  is the  $j_{\text{sup}.th}$  data entry and the corresponding label.

(24) Further, iterative processes of solving the sub-problems locally in step 3 are respectively:

(25) horizontal data distribution:

$$(26) \{v_i^{k+1}, v_i^{k+1}\} = \arg \min_{v_i} L(v_i - v_i^k) + \frac{r_i}{2} \text{Math. } v_i - v_i^k \text{Math. }^2, k \text{ forward. } v_i^{k+1}, v_{i,i+1}^{k+1} = v_{i,i+1}^k + v_{i,i+1}^{k+1} (v_i^{k+1} - v_{i+1}^{k+1});$$

and

(27) vertical data distribution

(28)

$$v_i^{k+1} = \arg \min_{v_i} f_i(v_i) + \frac{1}{2} \text{Math. } B_i v_i - B_i v_i^k - \bar{z}^k + Bv^k + u^k \text{Math. }^2, \bar{z}^{k+1} = \arg \min_{\bar{z}} g(N\bar{z}) + \frac{N}{2} \text{Math. } \bar{z} - Bv^{k+1} - u^k \text{Math. }^2, u^{k+1} = u^k + Bv^{k+1}$$

(29) Further, a method of using the Boolean sharing to split the penalty parameter in step 4 specifically includes: considering  $p_{\text{sup}.k} \text{ forward. } p_{\text{sup}.k+1}$  is a progressive increase and an upper bound is  $r_{\text{sub}.i}$ , obtaining an appropriate value through a comparison to update  $p$ , and dividing  $p_{\text{sub}.i, i+1.\text{sup}.k}$  into  $p_{\text{sub}.i, i+1.\text{sup}.k} = q_{\text{sub}.i, i+1.\text{sup}.k} + q_{\text{sub}.i+1, i.\text{sup}.k}$  to securely compute  $p_{\text{sub}.i, i+1.\text{sup}.k}$ ; wherein an  $i_{\text{sup}.th}$  participant provides  $q_{\text{sub}.i, i+1.\text{sup}.k}$  and  $q_{\text{sub}.i, i+1.\text{sup}.k+1}$ , and an  $(i+1)_{\text{sup}.th}$  participant provides  $q_{\text{sub}.i+1, i.\text{sup}.k}$  and  $q_{\text{sub}.i+1, i.\text{sup}.k+1}$ ; comparing  $q_{\text{sub}.i, i+1.\text{sup}.k} + q_{\text{sub}.i+1, i.\text{sup}.k}$  with  $q_{\text{sub}.i, i+1.\text{sup}.k+1} + q_{\text{sub}.i+1, i.\text{sup}.k+1}$  without exposing  $q_{\text{sub}.i, i+1.\text{sup}.k}$ ,  $q_{\text{sub}.i+1, i.\text{sup}.k}$ , and  $q_{\text{sub}.i+1, i.\text{sup}.k+1}$ , converting each term into a Boolean type, and performing a secure addition and comparison by using a Yao's garbled circuit.

(30) Further, a method of using the arithmetic sharing to split the penalty parameter in step 5 specifically includes: in a  $(k+1)_{\text{sup}.th}$  iteration, securely computing

$$(31) v_{i,i+1}^{k+1} (v_{i+1}^{k+1} - v_i^{k+1}) \text{ as } (q_{i,i+1}^{k+1} + q_{i+1,i}^{k+1}) (v_{i+1}^{k+1} - v_i^{k+1})$$

by using Shamir's secret sharing, and arithmetically dividing each term, wherein an  $i_{\text{sup}.th}$  participant provides   $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+1}$    $q_{\text{sub}.i, i+1.\text{sup}.k+$

machine privacy-preserving method.

(39) By means of the above technical solutions, the present invention has the following advantages. According to the present invention, the support vector machine for privacy-preserving is trained by combining an ADMM algorithm and the secret sharing. During a training process among the entities, the entities exchange part of the information divided by themselves for collaborative training. The present invention is based on an honest-but-curious model, in which all participating entities do not trust each other, and complete the training under the premise that individual information will not be leaked. Compared with the data processing method based on homomorphic encryption, the present invention has the features of simple computation and low computational overhead. Compared with the differential privacy method, the present invention provides cryptographically strong and secure privacy-preserving without affecting the accuracy of the training result.

(40) TABLE-US-00001 TABLE 1 Comparison between efficiencies of a multi-party secure computation scheme and a homomorphic encryption scheme

|  | FSCM-C | D | Q | Offline | Online | ADMIVI | Paillier | FSVM-S | 100 | 100 | 0.214 | 2.76E-4 | 0.0698 | 2.19E-4 | 200 | 200 | 0.810 | 3.10E-4 | 0.155 | 2.75E-4 | 300 | 300 | 1.83 | 3.33E-4 | 0.217 | 2.92E-4 | 400 | 400 | 3.24 | 3.70E-4 | 0.297 | 3.07E-4 | 500 | 500 | 5.13 | 4.01E-4 | 0.362 | 3.20E-4 | 600 | 600 | 7.46 | 4.16E-4 | 0.432 | 3.32E-4 | 700 | 700 | 10.02 | 4.53E-4 | 0.509 | 3.44E-4 | 800 | 800 | 13.21 | 4.82E-4 | 0.615 | 3.67E-4 | 900 | 900 | 16.60 | 5.08E-4 | 0.647 | 3.79E-4 | 1000 | 1000 | 20.56 | 5.35E-4 | 0.722 | 3.98E-4 |
|--|--------|---|---|---------|--------|--------|----------|--------|-----|-----|-------|---------|--------|---------|-----|-----|-------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|-------|---------|-------|---------|-----|-----|-------|---------|-------|---------|-----|-----|-------|---------|-------|---------|------|------|-------|---------|-------|---------|
|--|--------|---|---|---------|--------|--------|----------|--------|-----|-----|-------|---------|--------|---------|-----|-----|-------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|------|---------|-------|---------|-----|-----|-------|---------|-------|---------|-----|-----|-------|---------|-------|---------|-----|-----|-------|---------|-------|---------|------|------|-------|---------|-------|---------|

(41) In the present invention, the distributed support vector machine privacy-preserving method is based on a secure multi-party computation and an ADMM algorithm and, in an honest-but-curious scenario, uses the idea of data division to perform local computation through the interaction of part of the data among users, thereby finally reconstructing the secret to preserve data privacy. Since the whole data value of a single user is related to privacy information, after the data is divided, each collaborative user is allocated a part of the data for local computation. In this way, the partners entirely cannot get the relevant privacy information of other users, and all information with explicit semantics which the partner may obtain is only its own value and the final computed result.

## Description

### BRIEF DESCRIPTION OF THE DRAWINGS

(1) In order to explain the technical solutions of the embodiments of the present invention more clearly, the drawings used in the embodiments of the present invention will be briefly introduced below. Obviously, the drawings described below are only some embodiments of the present invention. Those of ordinary skill in the art can obtain other drawings based on these drawings without creative efforts.

(2) FIG. 1 is a flow chart of a distributed support vector machine privacy-preserving method according to an embodiment of the present invention.

(3) FIG. 2 is a structural schematic diagram of a distributed support vector machine privacy-preserving system according to an embodiment of the present invention, wherein 1 represents information preprocessing module; 2 represents information iterative processing module; and 3 represents privacy-preserving module.

(4) FIG. 3 is a schematic diagram of an application scenario according to an embodiment of the present invention.

(5) FIG. 4 is a schematic diagram of the implementation principle of the distributed support vector machine privacy-preserving method based on secure multi-party computation according to an embodiment of the present invention.

(6) FIG. 5 is a first schematic diagram of the collaborative training of two nodes of a breast cancer data set according to an embodiment of the present invention.

(7) FIG. 6 is a second schematic diagram of the collaborative training of the two nodes of the breast cancer data set according to an embodiment of the present invention.

### DETAILED DESCRIPTION OF THE EMBODIMENTS

(8) In order to make the objectives, technical solutions, and advantages of the present invention clearer, the present invention will be further described in detail below with reference to the embodiments. It should be understood that the specific embodiments described herein are only used to explain the present invention, rather than to limit the present invention.

(9) In view of the problems in the prior art, the present invention provides a distributed support vector machine privacy-preserving method, system, storage medium and application. The present invention will be described in detail below with reference to the drawings. Herein, local support vector machine sub-problems are solved by using a gradient descent method. Since the gradient descent method has a slow convergence rate and may converge to a local optimal solution, it may be replaced with improved methods such as damped Newton's method and variable metric method to solve the local sub-problems. In consideration of the real scenario, different entities can use different methods to solve the local sub-problems.

(10) As shown in FIG. 1, according to the present invention, a distributed support vector machine privacy-preserving method includes the following steps:

(11) S101: a network communication environment with a plurality of data sources is established;

(12) S102: a support vector machine scenario with a vertical distribution or a horizontal distribution is chosen according to a data distribution of the data sources;

(13) S103: all participating entities solve the sub-problems locally by using a gradient descent method;

(14) S104: all participating entities use Boolean sharing to split a penalty parameter and exchange a part of the penalty parameter with a neighboring node to update the parameter;

(15) S105: all participating entities use arithmetic sharing to split the updated iterative variable and exchange a part of the updated iterative variable with the neighboring node to compute a Lagrange parameter in a shared form;

(16) S106, all participating entities reconstruct the secret;

(17) S107, returning to S103 if the iteration does not converge; and

(18) S108, a training result is output.

(19) Those of ordinary skill in the art can also implement the distributed support vector machine privacy-preserving method by using other steps. FIG. 1 only illustrates a specific embodiment of the distributed support vector machine privacy-preserving method of the present invention.

(20) As shown in FIG. 2, according to the present invention, a distributed support vector machine privacy-preserving system includes:

(21) the information preprocessing module 1, configured for dividing a secret through secret sharing among all participating entities, iteratively exchanging a part of the information divided by the participating entities, and solving sub-problems locally;

(22) the information iterative processing module 2, configured for performing an iteration until a convergence is reached to find a global optimal solution; and

(23) the privacy-preserving module 3, configured for adopting a privacy-preserving method based on a vertical data distribution and a privacy-preserving method based on a horizontal data distribution, respectively; wherein the participating entities do not trust each other, and interact through a multi-party computation for local training.

(24) The technical solutions of the present invention will be further described below with reference to the drawings.

(25) According to the present invention, the distributed support vector machine privacy-preserving method includes: dividing a secret through secret sharing among all participating entities, iteratively exchanging a part of the information divided by the participating entities, and solving sub-problems locally; performing an iteration until a convergence is reached to find a global optimal solution; and in consideration of the generality of the privacy-preserving method, adopting a privacy-preserving method based on a vertical data distribution and a privacy-preserving method based on a horizontal data distribution, respectively; wherein the participating entities do not trust each other, and interact through a multi-party computation

for local training.

(26) As shown in FIG. 3, the application scenario of the present invention is a training process of a fully distributed multi-data source support vector machine, and the network includes users participating in the training. According to the network topology, the users who need to participate in the training set an initial value. In the iterative process, the value that needs to be computed collaboratively is divided through secret sharing and then exchanged for computation. Since the divided data cannot get any intermediate-state privacy information, data privacy and security are preserved.

(27) As shown in FIG. 4, according to an embodiment of the present invention, the distributed support vector machine privacy-preserving method based on secure multi-party computation specifically includes the following steps:

(28) Step 1: a network communication environment is established, and a network topology situation where multiple users are adjacent to each other is considered when the number of different users is set.

(29) Step 2: the iterative processes of solving the objective function for training the support vector machine are determined according to the data distribution of the data sources.

(30) Step 3: in the  $(k+1).sup.th$  iteration, the user first updates  $v.sub.i.sup.k+1$  according to the penalty function  $p$  and the Lagrange coefficient  $\lambda$  updated in the  $k.sup.th$  iteration.

(31) Step 4: in the  $(k+1).sup.th$  iteration, the user updates the penalty coefficient  $p$  by taking the progressive increase as a constraint condition.

$p.sub.i, i+1.sup.k$  is divided into  $p.sub.i, i+1.sup.k=q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k$  to securely compute  $p.sub.i, i+1.sup.k$ . The  $i.sup.th$  participant provides  $q.sub.i, i+1.sup.k$  and  $q.sub.i, i+1.sup.k+1$ , and the  $(i+1).sup.th$  participant provides  $q.sub.i+1, i.sup.k$  and  $q.sub.i+1, i.sup.k+1$ .  $q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k$  is compared with  $q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1$  without exposing  $q.sub.i, i+1.sup.k$ ,  $q.sub.i+1, i.sup.k$ ,  $q.sub.i, i+1.sup.k+1$ , and  $q.sub.i+1, i.sup.k+1$ . Each term is converted into a Boolean type, and is securely added and compared by using a Yao's garbled circuit. One party encrypts the truth table, one party performs circuit computation, and finally the secret is reconstructed. In this way, an appropriate penalty coefficient  $p$  is determined.

(32) Step 5: in the  $(k+1).sup.th$  iteration, the user solves the Lagrange coefficient  $\lambda.sub.i, i+1.sup.k+1$  by using the updated  $v.sub.i.sup.k+1$  and  $p.sub.i, i+1.sup.k$ , and securely computes  $p.sub.i, i+1.sup.k+1 (v.sub.i+1.sup.k+1-v.sub.i.sup.k+1)$  as  $(q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1) (v.sub.i+1.sup.k+1-v.sub.i.sup.k+1)$  by using Shamir's secret sharing. Each term is arithmetically divided. The  $i.sup.th$  participant provides  $\text{custom character } q.sub.i, i+1.sup.k+1 \text{ custom character.sub.1.sup.A, custom character.sub.i, i+1.sup.k+1 custom character.sub.2.sup.A, custom character-v.sub.i.sup.k+1 custom character.sub.1.sup.A, custom character-v.sub.i.sup.k+1 custom character.sub.2.sup.A, custom character-q.sub.i, i+1.sup.k+1 v.sub.i.sup.k+1 custom character.sub.1.sup.A, and custom character-q.sub.i, i+1.sup.k+1 v.sub.i.sup.k+1 custom character.sub.2.sup.A, and the (i+1).sup.th participant provides custom character } q.sub.i+1, i.sup.k+1 \text{ custom character.sub.1.sup.A, custom character } q.sub.i+1, i.sup.k+1 \text{ custom character.sub.2.sup.A, custom character v.sub.i+1.sup.k+1 custom character.sub.1.sup.A, custom character v.sub.i+1.sup.k+1 custom character.sub.2.sup.A, custom character } q.sub.i+1, i.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.1.sup.A, and custom character } q.sub.i+1, i.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.2.sup.A. The i.sup.th participant sends custom character } q.sub.i, i+1.sup.k+1 \text{ custom character.sub.2.sup.A, custom character-v.sub.i.sup.k+1 custom character.sub.2.sup.A and custom character-q.sub.i, i+1.sup.k+1 v.sub.i.sup.k+1 custom character.sub.2.sup.A to the (i+1).sup.th participant. The (i+1).sup.th participant sends custom character } q.sub.i+1, i.sup.k+1 \text{ custom character.sub.1.sup.A, custom character v.sub.i+1.sup.k+1 custom character.sub.1.sup.A and custom character } q.sub.i+1, i.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.1.sup.A to the i.sup.th participant. The i.sup.th participant locally computes custom character-q.sub.i+1, i.sup.k+1 v.sub.i.sup.k+1 custom character.sub.1.sup.A and custom character } q.sub.i, i+1.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.1.sup.A to finally determine the value of } (q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1) (v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) \text{ in the shared form as custom character } (q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1) (v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) \text{ custom character.sub.1.sup.A= custom character } q.sub.i, i+1.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.1.sup.A+ custom character } q.sub.i+1, i.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.1.sup.A+ custom character-q.sub.i+1, i.sup.k+1 v.sub.i.sup.k+1 custom character.sub.1.sup.A+ custom character-q.sub.i, i+1.sup.k+1 v.sub.i.sup.k+1 custom character.sub.1.sup.A. Similarly, the (i+1).sup.th participant computes custom character } (q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1) (v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) \text{ custom character.sub.2.sup.A= custom character } q.sub.i, i+1.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.2.sup.A+ custom character } q.sub.i+1, i.sup.k+1 v.sub.i+1.sup.k+1 \text{ custom character.sub.2.sup.A+ custom character-q.sub.i+1, i.sup.k+1 v.sub.i.sup.k+1 custom character.sub.2.sup.A+ custom character-q.sub.i, i+1.sup.k+1 v.sub.i.sup.k+1 custom character.sub.2.sup.A.}$

(33) Step 6: the interacting participating parties reconstruct the secret  $(q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k) (v.sub.i+1.sup.k-v.sub.i.sup.k)$  as  $(q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k) (v.sub.i+1.sup.k-v.sub.i.sup.k)= \text{custom character } (q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k) (v.sub.i+1.sup.k-v.sub.i.sup.k) \text{ custom character.sub.1.sup.A+ custom character } (q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k) (v.sub.i+1.sup.k-v.sub.i.sup.k) \text{ custom character.sub.2.sup.A, and compute } \lambda.sub.i, i+1.sup.k+1= \lambda.sub.i, i+1.sup.k+ p.sub.i, i+1.sup.k+1 (v.sub.i.sup.k+1-v.sub.i+1.sup.k+1) \text{ to update } \lambda.sub.i, i+1.sup.k+1$ .

(34) Step 7: according to a set threshold  $E$ , when a value of the objective function at a current iteration minus a value of the objective function at a previous iteration is less than the threshold, it is determined that the convergence is reached. Otherwise, returning to step 3 to continue the iteration.

(35) Step 8: a training result is output.

(36) The effectiveness of the present invention will be further described below in conjunction with experiments.

#### 1. Experimental Conditions

(37) The experiment is simulated under Ubuntu-18.04.1, and the function of secure multi-party computation is implemented by using an ABY framework. The privacy-preserving scheme is implemented by c++.

#### 2. Experimental Results and Analysis

(38) In the present invention, Ubuntu is selected for simulation. The MNIST data set and the breast cancer data set are selected for a test. 2, 3, 4, 5, and 6 nodes are selected to perform a horizontal distribution experiment and a vertical distribution experiment, respectively. In the simulation experiment, the classification accuracy of the support vector machine is 98%.

(39) In the experiment, the established network communication model faces the threat of data privacy leakage. Different users collaborate to train the support vector machine, and the intermediate state of the interaction during the training process will leak privacy information such as gradient and objective function. The development of distributed scenarios brings an increasing amount of data. In order to break data silos and carry out certain collaborative training scenarios, a feasible privacy-preserving method is indispensable. The prior distributed support vector machines have mutually reciprocal shortcomings between computational overhead and security. That is, a high-security method has the problem of high computational overhead, whereas a high-efficiency method has security issues. In addition, such methods must give consideration to both the machine learning scenario and the accuracy of the training results. In the present invention, the support vector machine for privacy-preserving is trained by combining the ADMM algorithm and the secret sharing. During the training process among the entities, the entities exchange part of the information divided by themselves for collaborative training. The present invention is based on an honest-but-curious model, in which all participating entities do not trust each other, and complete the training under the premise that individual information will not be leaked.

(40) FIG. 5 illustrates the collaborative training of two nodes of a breast cancer data set, in which the classification accuracy of the support vector machine based on the privacy-preserving ADMM algorithm in the horizontal data distribution scenario is 98.2%, while the classification accuracy in the case where only one node uses the gradient descent method to solve the optimization problem is also 98%. Therefore, it can be known that the method of the present invention provides cryptographically strong and secure privacy-preserving without affecting the accuracy of the training results.

(41) FIG. 6 illustrates the collaborative training of the two nodes of the breast cancer data set. The classification accuracy of the support vector machine based on the privacy-preserving ADMM algorithm in the vertical data distribution scenario is 97.7%. When the vertical distribution is applied to an actual scenario, different entities can have data sets with different features, and collaborate to train a global classification model. (42) It should be noted that the embodiments of the present invention can be implemented by hardware, software, or a combination of software and hardware. The hardware part can be implemented by dedicated logic. The software part can be stored in a memory, and the system can be executed by appropriate instructions, for example, the system can be executed by a microprocessor or dedicated hardware. Those of ordinary skill in the art can understand that the above-mentioned devices and methods can be implemented by using computer-executable instructions and/or control codes included in a processor. Such codes are provided, for example, on a carrier medium such as a magnetic disk, compact disc (CD) or digital video disk read-only memory (DVD-ROM), a programmable memory such as a read-only memory (firmware), or a data carrier such as an optical or electronic signal carrier. The device and its modules of the present invention can be implemented by very large-scale integrated circuits or gate arrays, semiconductors such as logic chips and transistors, or programmable hardware devices such as field programmable gate arrays and programmable logic devices, and other hardware circuits. Optionally, the device and its modules of the present invention can be implemented by software executed by various types of processors, or can be implemented by a combination of the hardware circuit and the software as mentioned above, such as firmware.

(43) The above only describes the specific embodiments of the present invention, but the scope of protection of the present invention is not limited thereto. Any modifications, equivalent replacements, improvements and others made by any person skilled in the art within the technical scope disclosed in the present invention and the spirit and principle of the present invention shall fall within the scope of protection of the present invention.

## Claims

1. A distributed support vector machine privacy-preserving method, wherein the method is performed by a distributed support vector machine, and the method comprising: dividing a secret through a secret sharing among a plurality of participating machines, iteratively exchanging a part of information divided by the plurality of participating machines, and solving sub-problems locally at each of the plurality of participating machines; performing an iteration until a convergence is reached to find a global optimal solution; and in consideration of generality of the distributed support vector machine privacy-preserving method, adopting the distributed support vector machine privacy-preserving method based on a vertical data distribution and the distributed support vector machine privacy-preserving method based on a horizontal data distribution, respectively, wherein the plurality of participating machines interact through a multi-party computation for local training at each of the plurality of participating machines; step 1: establishing, by a processor, a network communication environment with a plurality of data sources; step 2: choosing, by the processor, a support vector machine scenario with a vertical distribution or a horizontal distribution according to a data distribution of the plurality of data sources; step 3: solving, by the plurality of participating machines, the sub-problems locally at each of the plurality of participating machines; step 4: splitting, by the plurality of participating machines, a penalty parameter using a Boolean sharing and exchanging, by the plurality of participating machines, a part of the penalty parameter with a neighboring node to update the penalty parameter; step 5: splitting, by the plurality of participating machines, an updated iterative variable using an arithmetic sharing and exchanging, by the plurality of participating machines, a part of the updated iterative variable with the neighboring node to compute a Lagrange parameter in a shared form; step 6, reconstructing, by the plurality of participating machines, the secret; step 7, returning to step 3 if the iteration does not reach the convergence; and step 8, outputting, by the processor, a training result; wherein an objective function of the horizontal distribution and an objective function of the vertical distribution in step 2 are respectively:

$$\min_{v_i} \frac{1}{2} \sum_{i=1}^N \text{Math. } v_i^T (I_{D+1} - D_{+1}) v_i + NC \sum_{i=1}^N \text{Math. } 1_i^T s_i \cdot t. \quad \{ Y_i B_i V_i \geq 1_i - i, i \geq 0_i (i = 1, \text{Math.}, N); \quad \text{wherein } i \text{ refers to the } i.\text{sup.th participant; } M.\text{sub.i is the number of training data entries in the } i.\text{sup.th participant's local training dataset; } N \text{ is the number of participants; } D \text{ refers to a number of attributes of each data entry in the dataset; } C \text{ refers to the penalty parameter; } T \text{ refers to transpose operation in math; } \omega \text{ stands for a normal vector of the hyperplane and } b \text{ is an interception of the hyperplane with a y-axis; for a more concise description of the above equation, we introduce a column vector } v.\text{sub.i of the form } v.\text{sub.i}=[(\omega.\text{sub.i}.\text{sup.T}, b.\text{sub.i}.\text{sup.T}), \text{sup.T}, \text{ to stands for the transpose of the vector } \omega.\text{sub.i}.\text{sup.T plus one element } b.\text{sub.i; the term } V, \text{ where } V=[(v.\text{sub.i}.\text{sup.T}, \dots v.\text{sub.N}.\text{sup.T}), \text{sup.T}, \text{ is a column vector that stores all the elements of the vectors } v.\text{sub.i}(i=1, \dots, N) \text{ and this term is also used to make the above equation concise; } R \text{ refers to the set of real numbers and } R.\text{sup.1} * M.\text{sub.i is the set of all the row vectors of dimension } M.\text{sub.i whose elements are real numbers; } X.\text{sub.i is the matrix of dimension } M.\text{sub.i} \times D, \text{ that stores all the training data vectors of the } i.\text{sup.th participant and its form is } X.\text{sub.i}=[x.\text{sub.i}.\text{sup.T}, \dots x.\text{sub.i}.\text{sup.T}]; \text{ the term } B.\text{sub.i, of the form } B.\text{sub.i}=[X.\text{sub.i}, 1.\text{sub.i}.\text{sup.T}], \text{ is the matrix that contains } X.\text{sub.i plus one column vector whose elements are all 1; } y.\text{sub.i.j is the corresponding label of the } j.\text{sup.th data entry in the } i.\text{sup.th participant's local training dataset; } Y.\text{sub.i}=\text{diag}(y.\text{sub.i}.\text{sup.T}, \dots, y.\text{sub.i}.\text{sup.T}) \text{ is a matrix of dimension } M.\text{sub.i} \times M.\text{sub.i, whose diagonal elements are } y.\text{sub.i.j}(j=1, \dots, M.\text{sub.i}) \text{ and other elements are all 0; } \xi.\text{sub.i}=[\xi.\text{sub.i}.\text{sup.T}, \dots, \xi.\text{sub.i}.\text{sup.T}] \text{ is the vector that stores the slack variables for each data entry of the } i.\text{sup.th participant; and}$$

$$\min_{v_i} \sum_{i=1}^N \frac{1}{2} v_i^T (I_{D+1} - D_{+1}) v_i + \sum_{i=1}^N \text{Math. } B_i v_i \cdot t. \quad z_i = B_i v_i, i = 1, \text{Math.}, N; \quad \text{wherein } v.\text{sub.i}=[\omega.\text{sub.i}.\text{sup.T}, b.\text{sub.i}.\text{sup.T}], \text{sup.T}, B.\text{sub.i}=[X.\text{sub.i}, 1.\text{sub.i}.\text{sup.T}], 1.\text{sub.i}.\text{sup.T} \in R.\text{sup.M} * 1, Y.\text{sub.i}=\text{diag}(y.\text{sub.i}.\text{sup.T}, \dots, y.\text{sub.i}.\text{sup.T}), y.\text{sub.i.j is the } j.\text{sup.th data entry and the corresponding label.}$$

2. The distributed support vector machine privacy-preserving method according to claim 1, wherein iterative processes of solving the sub-problems locally in step 3 are respectively: for the horizontal data distribution:

$$\{v_i^{k+1}, \frac{k+1}{i}\} = \arg \min_{v_i} L(v_i^k - \bar{z}^k + \bar{v}^k + u^k) + \frac{r_i}{2} \text{Math. } v_i - v_i^k \text{Math. }^2, \quad \bar{z}^{k+1} = \bar{z}^k + \frac{r_i}{2} (v_i^{k+1} - v_i^k); \quad \text{and for the vertical data distribution:}$$

$$v_i^{k+1} = \arg \min_{v_i} f_i(v_i) + \frac{r_i}{2} \text{Math. } B_i v_i - B_i v_i^k - \bar{z}^k + \bar{v}^k + u^k \text{Math. }^2, \quad \bar{z}^{k+1} = \arg \min_{\bar{z}} g(N \bar{z}) + \frac{N}{2} \text{Math. } \bar{z} - \bar{v}^{k+1} - u^k \text{Math. }^2, \quad u^{k+1} = u^k + \bar{v}^k - \bar{v}^{k+1}$$

wherein k refers to k.sup.th iteration, z is a variable to formulate an ADMM objective function of a sharing form, f is a general denotation of the function to form the objective function that would be optimized by the ADMM algorithm, q refers to the arithmetic share of p, ξ refers to a slack variable, and u is the scaled dual variable.

3. The distributed support vector machine privacy-preserving method according to claim 2, wherein a method of using the Boolean sharing to split the penalty parameter in step 4 specifically comprises: considering p.sup.k.fwdarw.p.sup.k+1 is a progressive increase and an upper bound is r.sub.i, obtaining a value of the penalty parameter p through a comparison to update the penalty parameter p, and dividing p.sub.i, i+1.sup.k into p.sub.i, i+1.sup.k=q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k to securely compute p.sub.i, i+1.sup.k; wherein an i.sup.th participant provides q.sub.i, i+1.sup.k and q.sub.i, i+1.sup.k+1, and an (i+1).sup.th participant provides q.sub.i+1, i.sup.k and q.sub.i+1, i.sup.k+1; comparing q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k with q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k+1 without exposing q.sub.i, i+1.sup.k, q.sub.i+1, i.sup.k, q.sub.i, i+1.sup.k+1, and q.sub.i+1, i.sup.k+1, converting each term into a Boolean type, and performing a secure addition and the comparison by using a Yao's garbled circuit.

4. The distributed support vector machine privacy-preserving method according to claim 3, wherein a method of using the arithmetic sharing to split the penalty parameter in step 5 specifically comprises: in a (k+1).sup.th iteration, securely computing p.sub.i, i+1.sup.k+1(v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) as (q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1)(v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) by using Shamir's secret sharing, and arithmetically dividing each term, wherein an i.sup.th participant provides custom characterq.sub.i, i+1.sup.k+1



custom character.sub.1.sup.A, i+1.sup.k+1 custom character.sub.2.sup.A, custom character-v.sub.i.sup.k+1 custom character.sub.1.sup.A, custom character-q.sub.i, i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.1.sup.A, and custom character-q.sub.i, i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.2.sup.A, an (i+1).sup.th participant provides custom characterq.sub.i+1, i.sup.k+1 custom character.sub.1.sup.A, custom character q.sub.i+1, i.sup.k+1 custom character.sub.2.sup.A, custom characterv.sub.i+1.sup.k+1 custom character.sub.1.sup.A, custom character v.sub.i+1.sup.k+1 custom character.sub.2.sup.A, custom characterq.sub.i+1, i.sup.k+1v.sub.i+1.sup.k+1 custom character.sub.1.sup.A, and custom characterq.sub.i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.2.sup.A, the i.sup.th participant sends custom characterq.sub.i, i+1.sup.k+1 custom character.sub.2.sup.A, custom character-v.sub.i.sup.k+1 custom character.sub.2.sup.A and custom character-q.sub.i, i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.2.sup.A to the (i+1).sup.th participant, the (i+1).sup.th participant sends custom character q.sub.i+1, i.sup.k+1 custom character.sub.1.sup.A, custom characterv.sub.i+1.sup.k+1 custom character.sub.1.sup.A and custom character q.sub.i+1, i.sup.k+1v.sub.i+1.sup.k+1 custom character.sub.1.sup.A to the i.sup.th participant, and the i.sup.th participant locally computes custom character-q.sub.i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.1.sup.A and custom characterq.sub.i, i+1.sup.k+1v.sub.i+1.sup.k+1 custom character.sub.1.sup.A to finally determine a value of (q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1)(v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) in the shared form as custom character(q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1)(v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) custom character .sub.1.sup.A= custom characterq.sub.i, i+1.sup.k+1v.sub.i+1.sup.k+1 custom characterq.sub.i+1.sup.k+1v.sub.i+1.sup.k+1 custom character .sub.1.sup.A+ custom character-q.sub.i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.1.sup.A+ custom character-q.sub.i, i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.1.sup.A; and the (i+1).sup.th participant computes custom character(q.sub.i, i+1.sup.k+1+q.sub.i+1, i.sup.k+1)(v.sub.i+1.sup.k+1-v.sub.i.sup.k+1) custom character.sub.2.sup.A= custom characterq.sub.i, i+1.sup.k+1v.sub.i+1.sup.k+1 custom character.sub.2.sup.A+ custom characterq.sub.i+1, i.sup.k+1v.sub.i+1.sup.k+1 custom character .sub.2.sup.A+ custom character-q.sub.i+1, i.sup.k+1v.sub.i.sup.k+1 custom character.sub.2.sup.A+ custom character-q.sub.i, i+1.sup.k+1v.sub.i.sup.k+1 custom character.sub.2.sup.A.

5. The distributed support vector machine privacy-preserving method according to claim 4, wherein a method of reconstructing the secret in step 6 specifically comprises: allowing the plurality of participating machines to reconstruct the secret (q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k) (v.sub.i+1.sup.k-v.sub.i.sup.k) as (q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k)(v.sub.i+1.sup.k-v.sub.i.sup.k)= custom character(q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k)(v.sub.i+1.sup.k-v.sub.i.sup.k) custom character.sub.1.sup.A+ custom character(q.sub.i, i+1.sup.k+q.sub.i+1, i.sup.k) (v.sub.i+1.sup.k-v.sub.i.sup.k) custom character.sub.2.sup.A, and compute  $\lambda_{sub.i}, i+1.sup.k+1=\lambda_{sub.i}, i+1.sup.k+p_{sub.i}, i+1.sup.k+1(v_{sub.i}, i+1.sup.k+1-v_{sub.i}, i+1.sup.k+1)$  to update the Lagrange parameter  $\lambda$ .

6. A computer-readable storage medium, wherein a computer program is stored in the computer-readable storage medium, and when the computer program is executed by a processor, the processor executes the following steps: dividing a secret through a secret sharing among a plurality of participating machines, iteratively exchanging a part of information divided by the plurality of participating machines, and solving sub-problems locally at each of the plurality of participating machines; performing an iteration until a convergence is reached to find a global optimal solution; in consideration of generality of a distributed support vector machine privacy-preserving method, adopting the distributed support vector machine privacy-preserving method based on a vertical data distribution and the distributed support vector machine privacy-preserving method based on a horizontal data distribution, respectively; wherein the plurality of participating machines interact through a multi-party computation for local training at each of the plurality of participating machines; step 1: establishing, by a processor, a network communication environment with a plurality of data sources; step 2: choosing, by the processor, a support vector machine scenario with a vertical distribution or a horizontal distribution according to a data distribution of the plurality of data sources; step 3: solving, by the plurality of participating machines, the sub-problems locally at each of the plurality of participating machines; step 4: splitting, by the plurality of participating machines, a penalty parameter using a Boolean sharing and exchanging, by the plurality of participating machines, a part of the penalty parameter with a neighboring node to update the penalty parameter; step 5: splitting, by the plurality of participating machines, an updated iterative variable using an arithmetic sharing and exchanging, by the plurality of participating machines, a part of the updated iterative variable with the neighboring node to compute a Lagrange parameter in a shared form; step 6, reconstructing, by the plurality of participating machines, the secret; step 7, returning to step 3 if the iteration does not reach the convergence; and step 8, outputting, by the processor, a training result; wherein an objective function of the horizontal distribution and an objective function of the vertical distribution in step 2 are respectively:  $\min_{v_i} \frac{1}{2} \sum_{i=1}^N \text{Math. } v_i^T (I_{D+1} - I_{D+1}) v_i + NC \sum_{i=1}^N 1_i^T$

$s.t. \begin{cases} Y_i B_i V_i \geq 1_i - \epsilon_i, & \epsilon_i \geq 0 (i = 1, \dots, N); \end{cases}$  wherein i refers to the i.sup.th participant; M.sub.i is the number of training data entries in the i.sup.th participant's local training dataset; N is the number of participants; D refers to a number of attributes of each data entry in the dataset; C refers to the penalty parameter; T refers to transpose operation in math;  $\omega$  stands for a normal vector of the hyperplane and b is an interception of the hyperplane with a y-axis; for a more concise description of the above equation, we introduce a column vector v.sub.i of the form v.sub.i=[( $\omega_{sub.i}, i, sup.T, b_{sub.i}$ ).sup.T, to stands for the transpose of the vector  $\omega_{sub.i}, i, sup.T$  plus one element  $b_{sub.i}$ ; the term V, where  $V=[(v_{sub.i}, i, sup.T, \dots, v_{sub.N}, sup.T).sup.T$ , is a column vector that stores all the elements of the vectors v.sub.i(i=1, ..., N) and this term is also used to make the above equation concise; R refers to the set of real numbers and  $R_{sup.1 \times M_{sup.i}}$  is the set of all the row vectors of dimension M.sub.i whose elements are real numbers; X.sub.i is the matrix of dimension M.sub.i×D, that stores all the training data vectors of the i.sup.th participant and its form is  $X_{sub.i}=[x_{sub.i}, i, sup.T, \dots, x_{sub.i}, M_{sup.T}]$ ; the term B.sub.i, of the form  $B_{sub.i}=[X_{sub.i}, 1_{sub.i}, sup.T]$ , is the matrix that contains X.sub.i plus one column vector whose elements are all 1; y.sub.ij is the corresponding label of the j.sup.th data entry in the i.sup.th participant's local training dataset;  $Y_{sub.i}=\text{diag}(y_{sub.i}, 1, \dots, y_{sub.i}, M_{sub.i})$  is a matrix of dimension M.sub.i×M.sub.i, whose diagonal elements are y.sub.ij(j=1, ..., M.sub.i) and other elements are all 0;  $\xi_{sub.i}=[\xi_{sub.i}, 1, \dots, \xi_{sub.i}, M_{sub.i}]$  is the vector that stores the slack variables for each data entry of the i.sup.th participant; and  $\min \sum_{i=1}^N \frac{1}{2} v_i^T (I_{D+1} - I_{D+1}) v_i + 1_M^T (1_M - Y \sum_{i=1}^N B_i v_i)$   $s.t. \quad z_i = B_i v_i, i = 1, \dots, N$ ; wherein v.sub.i=

$[\omega_{sub.i}, i, sup.T, b_{sub.i}.sup.T, B_{sub.i}=[X_{sub.i}, 1_{sub.i}, sup.T], 1_{sub.i} \in R_{sup.M \times 1}, Y_{sub.i}=\text{diag}(y_{sub.i}, 1, \dots, y_{sub.i}, M), y_{sub.j}$  is the j.sup.th data entry and the corresponding label.

7. A distributed support vector machine privacy-preserving system for implementing the distributed support vector machine privacy-preserving method according to claim 1, comprising: an information preprocessing module, configured for dividing the secret through the secret sharing among the plurality of participating machines, iteratively exchanging the part of the information divided by the plurality of participating machines, and solving the sub-problems locally; an information iterative processing module, configured for performing the iteration until the convergence is reached to find the global optimal solution; and a privacy-preserving module, configured for adopting the distributed support vector machine privacy-preserving method based on the vertical data distribution and the distributed support vector machine privacy-preserving method based on the horizontal data distribution, respectively.

8. A distributed support vector machine for implementing the distributed support vector machine privacy-preserving method according to claim 1.

9. The distributed support vector machine privacy-preserving system according to claim 7, wherein iterative processes of solving the sub-problems locally in step 3 are respectively: for the horizontal data distribution:

$\{v_i^{k+1}, \dots, v_i^{k+1}\} = \arg \min_{v_i} L(v_i^{k+1} - v_i^k) + \frac{r_i}{2} \text{Math. } v_i - v_i^k \text{Math. }^2, \dots, v_i^{k+1}, \dots, v_{i,i+1}^{k+1} = \frac{k}{i, i+1} + \frac{k+1}{i, i+1} (v_i^{k+1} - v_{i+1}^{k+1})$ ; and for the vertical data distribution:

